JavaScript

Introduction to Software Systems

Web Dev 101

01

JavaScript Basics

The brain and muscle of the web. If HTML is the skeleton and CSS is the skin, JavaScript is what allows the page to move, think, and react.

Adding JS & Variables

Adding JavaScript

Always link your JS file right before the closing body tag inside your HTML.

```
src="app.js">
```

Variables

Where we store and update our data.

```
// app.js
let searchCount = 0;
// 'let' can be changed later

const siteName = "Google";
// 'const' cannot change (constant)
```

Data Types & Logic

Data Types

JS understands different kinds of data natively.

```
let query = "web dev"; // String
let results = 104000;  // Number
let isDark = false;    // Boolean
```

If / Else Logic

Making decisions based on true/false conditions.

```
if (query === "") {
  console.log("Empty!");
} else {
  search(query);
}
```

Arrays & Loops

Arrays (Lists)

Arrays let you store multiple values in a single variable. We access them using an index (starting at 0).

```
let history = ["html", "css", "js"];

console.log(history[0]); // "html"
console.log(history.length); // 3

// Adding to the end
history.push("react");
```

Loops

Loops allow you to run the same code multiple times, perfectly paired with Arrays.

```
// A standard 'for' loop
for (let i = 0; i < history.length; i++) {
  console.log("I learned " + history[i]);
}

/* Output:
I learned html
I learned css
I learned js
I learned react */
```

Objects: Grouping Data

- **Key-Value Pairs**

  Objects are like dictionaries. You store data in a property (the key) and give it a value.

- **Real-World Modeling**

  Everything in JS is technically an object. It's how we model real-world things (like a user, a product, or a post).

- **Dot Notation**

  You can access the data inside an object using a simple dot: `user.name`

```
let searchResult = {
  title: "JavaScript Basics",
  url: "https://developer.mozilla.org",
  views: 45000,
  isSecure: true
};

// Reading values
console.log(searchResult.title);

// Updating values
searchResult.views = 45001;
```

02

Functions

Reusable blocks of code. Instead of writing the same logic over and over, we package it into a command we can call anytime.

Anatomy of a Function

- **Write Once, Run Anywhere**

  Package your code for later use. This keeps your scripts clean and modular.

- **Parameters**

  Functions can accept input values (parameters) to make them dynamic and adaptable.

- **Return Values**

  A function can process data and send a result back using the `return` keyword.

```
function greetUser(name) {
  return "Hello, " + name + "!";
}

const msg = greetUser("Student");
```

Theme Demo

```
function findSequence(goal) {
  function find(start, history) {
    if (start == goal)
      return history;
    else if (start > goal)
      return null;
    else
      return find(start + 5, "(" + history + " + 5)") ||
             find(start * 3, "(" + history + " * 3)");
  }
  return find(1, "1");
}
```

Select a theme: mbo

03

Events & The DOM

The Document Object Model (DOM) is your live HTML tree. We use JS to find elements, listen to user interactions, and modify the page.

1. Selecting Elements

Query Selector

The modern, easiest way to find elements. It uses exact same syntax as your CSS selectors!

```
// Selects the FIRST matching element
const btn = document.querySelector(".submit-
btn");
const title = document.querySelector("h1");

// Selects ALL matches (Returns list)
const links = document.querySelectorAll("a");
```

Get Element By ID

The traditional way to find a single, specific element on the page using its unique ID.

```
// Selects
const searchBar =
document.getElementById("search-bar");

// Notice: No '#' is needed here!
// The method already knows it's an ID.
```

2. Modifying Elements

Text & Content

Change the words on the screen.

```
const heading = document.
  querySelector("h1");

// Replaces the text
heading.textContent = "New Title!";
```

Inline Styles

Change CSS directly (use sparingly).

```
const box = document.
  querySelector(".box");

// camelCase for properties!
box.style.backgroundColor = "red";
box.style.fontSize = "20px";
```

Classes (Best Practice)

Add/remove classes to trigger CSS.

```
const menu = document.
  querySelector("nav");

menu.classList.add("hidden");
menu.classList.remove("active");
menu.classList.toggle("dark");
```

3. Listening for Actions

- **addEventListener()**

  This method attaches a "listener" to an element. It waits patiently for a specific user action to occur.

- **The Event Type**

  The first argument is the event name: `"click"`, `"input"`, `"keydown"`, `"submit"`.

- **The Callback Function**

  The second argument is the function that runs *only when* the event happens.

```
const btn = document.querySelector("button");

// 1. Element  2. Event  3. Function
btn.addEventListener("click", function() {

  console.log("Button was clicked!");
  btn.classList.toggle("active-state");

});
```

Debugging 101

- **The Console is Your Best Friend**

  Right click anywhere -> Inspect -> Console. This is where your JS talks to you.

- **`console.log()` Everything**

  If something isn't working, print out your variables! Verify that your code is actually running.

- **Reading Red Errors**

  Don't panic! JS tells you exactly what line broke.
  `ReferenceError` = Typo in variable name.
  `SyntaxError` = Missing a bracket or comma.

```
> console.log("Testing ... ")
Testing ...

> let myVar = document.querySlector("h1");
Uncaught TypeError:
document.querySlector is not a function
    at app.js:14:24


// Ah! I spelled 'querySelector' wrong
// and the error is on line 14!
```

04

In-Class Assignment

Build a Live Character Counter & Validator.

Take your Google Search Clone from Lab 6 and make the search bar smart.

What You'll Build

1. Page structure

Add a `0/50 characters` below your search input.

2. JavaScript Selection

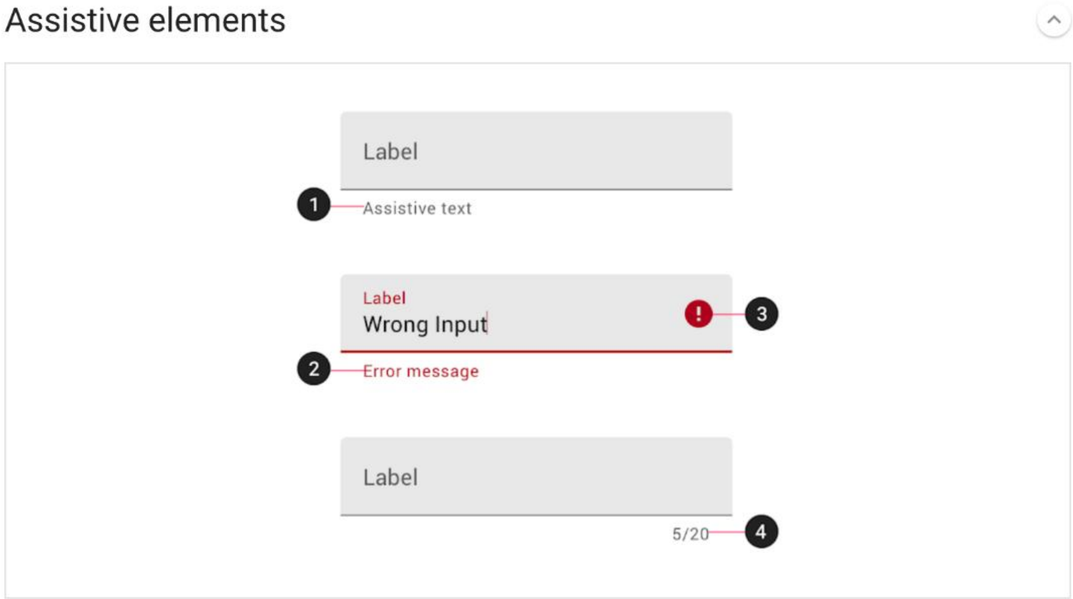
Select the input field, the character count paragraph, and the submit button using `document.querySelector()`.

3. Live Counting

Listen for the "input" event. As the user types, update the paragraph to show the current character length.

4. Form Validation

If the character count is 0, the "Google Search" button should be disabled. If > 0, the button should be clickable.



Assistive elements provide additional detail about text entered into text fields.

Tips & Hints

String Length

Get the length of text easily.

```
const txt = "Hello";
console.log(txt.length);
// Outputs: 5
```

Reading Values

Get what the user typed.

```
const inp = document.
  querySelector("input");

let len = inp.value.length;
```

Disable Buttons

Turn buttons off using JS.

```
const btn = document.
  querySelector("button");

btn.disabled = true;
```

Key Takeaways

Variables & Data

Use `let` for variables that change, and `const` for ones that don't. JS handles strings, numbers, booleans, arrays, and objects.

Functions

Functions package logic into reusable blocks. Write once, run anywhere!

The DOM is your playground

Use `document.querySelector()` to grab elements, and modify them using `.textContent`, `.style`, or `.classList`.

Event Listeners

`addEventListener` is the bridge between the user and your code. It makes your static HTML come alive in real-time.

Open your editor.

The best way to learn JavaScript is to break things and fix them.

Use your browser's Developer Console (Right Click -> Inspect -> Console) to check for errors!

Good luck with the assignment!